

Online Complaint Registration System

Detailed Project Report (Sample)

1. Introduction

The Online Complaint Registration System is a full-stack web application developed using the MERN stack. It provides a digital platform where users can create an account, authenticate themselves, and submit complaints from any location. The system replaces manual complaint books with a centralized web application that stores complaint information securely in a MongoDB database. This improves accessibility, transparency, and efficiency while reducing paperwork and delays. The application demonstrates frontend development, backend API development, database integration, and deployment. The Online Complaint Registration System is a full-stack web application developed using the MERN stack. It provides a digital platform where users can create an account, authenticate themselves, and submit complaints from any location. The system replaces manual complaint books with a centralized web application that stores complaint information securely in a MongoDB database. This improves accessibility, transparency, and efficiency while reducing paperwork and delays. The application demonstrates frontend development, backend API development, database integration, and deployment. The Online Complaint Registration System is a full-stack web application developed using the MERN stack. It provides a digital platform where users can create an account, authenticate themselves, and submit complaints from any location. The system replaces manual complaint books with a centralized web application that stores complaint information securely in a MongoDB database. This improves accessibility, transparency, and efficiency while reducing paperwork and delays. The application demonstrates frontend development, backend API development, database integration, and deployment. The Online Complaint Registration System is a full-stack web application developed using the MERN stack. It provides a digital platform where users can create an account, authenticate themselves, and submit complaints from any location. The system replaces manual complaint books with a centralized web application that stores complaint information securely in a MongoDB database. This improves accessibility, transparency, and efficiency while reducing paperwork and delays. The application demonstrates frontend development, backend API development, database integration, and deployment.

2. Need for the Project

Many organizations still rely on manual complaint handling. Paper records can be misplaced, searching is difficult, and tracking complaint progress is time-consuming. A web-based solution enables users to register complaints instantly, while administrators can maintain organized records. Digital storage also improves reporting and future scalability. Many organizations still rely on manual complaint handling. Paper records can be misplaced, searching is difficult, and tracking complaint progress is time-consuming. A web-based solution enables users to register complaints instantly, while administrators can maintain organized records. Digital storage also improves reporting and future scalability. Many organizations still rely on manual complaint handling. Paper records can be misplaced, searching is difficult, and tracking complaint progress is time-consuming. A web-based solution enables users to register complaints instantly, while administrators can maintain organized records. Digital storage also improves reporting and future scalability. Many organizations still rely on manual complaint handling. Paper records can be misplaced, searching is difficult, and tracking complaint progress is time-consuming. A web-based solution enables users to register complaints instantly, while administrators can maintain organized records. Digital storage also improves reporting and future scalability.

3. Objectives

The major objectives are to provide secure user registration and login, enable online complaint submission, store complaint information safely in MongoDB, create REST APIs using Express.js, build an interactive React user interface, and deploy the application online for easy access. The major objectives are to provide secure user registration and login, enable online complaint

submission, store complaint information safely in MongoDB, create REST APIs using Express.js, build an interactive React user interface, and deploy the application online for easy access. The major objectives are to provide secure user registration and login, enable online complaint submission, store complaint information safely in MongoDB, create REST APIs using Express.js, build an interactive React user interface, and deploy the application online for easy access. The major objectives are to provide secure user registration and login, enable online complaint submission, store complaint information safely in MongoDB, create REST APIs using Express.js, build an interactive React user interface, and deploy the application online for easy access.

4. Technologies

React.js is used to build reusable user interface components. Node.js provides the runtime environment. Express.js is used to build REST APIs. MongoDB stores application data. Mongoose provides object modeling. Git and GitHub are used for version control. Vercel is used to deploy the frontend. VS Code is the primary development environment. React.js is used to build reusable user interface components. Node.js provides the runtime environment. Express.js is used to build REST APIs. MongoDB stores application data. Mongoose provides object modeling. Git and GitHub are used for version control. Vercel is used to deploy the frontend. VS Code is the primary development environment. React.js is used to build reusable user interface components. Node.js provides the runtime environment. Express.js is used to build REST APIs. MongoDB stores application data. Mongoose provides object modeling. Git and GitHub are used for version control. Vercel is used to deploy the frontend. VS Code is the primary development environment. React.js is used to build reusable user interface components. Node.js provides the runtime environment. Express.js is used to build REST APIs. MongoDB stores application data. Mongoose provides object modeling. Git and GitHub are used for version control. Vercel is used to deploy the frontend. VS Code is the primary development environment.

5. System Architecture

The user accesses the React application through a web browser. When a user registers or submits a complaint, React sends HTTP requests to the Express server. The server validates the request, executes business logic through controllers, and communicates with MongoDB using Mongoose. The database stores or retrieves records, and the server returns JSON responses to the frontend for display. The user accesses the React application through a web browser. When a user registers or submits a complaint, React sends HTTP requests to the Express server. The server validates the request, executes business logic through controllers, and communicates with MongoDB using Mongoose. The database stores or retrieves records, and the server returns JSON responses to the frontend for display. The user accesses the React application through a web browser. When a user registers or submits a complaint, React sends HTTP requests to the Express server. The server validates the request, executes business logic through controllers, and communicates with MongoDB using Mongoose. The database stores or retrieves records, and the server returns JSON responses to the frontend for display. The user accesses the React application through a web browser. When a user registers or submits a complaint, React sends HTTP requests to the Express server. The server validates the request, executes business logic through controllers, and communicates with MongoDB using Mongoose. The database stores or retrieves records, and the server returns JSON responses to the frontend for display.

6. Modules

Registration Module: creates a new user account. Login Module: authenticates the user. Complaint Module: accepts complaint details and stores them. Database Module: manages CRUD operations. Deployment Module: hosts the application online for public access. Registration Module: creates a new user account. Login Module: authenticates the user. Complaint Module: accepts complaint details and stores them. Database Module: manages CRUD operations. Deployment Module: hosts the application online for public access. Registration Module: creates a new user account. Login Module: authenticates the user. Complaint Module: accepts complaint details and stores them. Database Module: manages CRUD operations. Deployment Module: hosts the application online for public access. Registration Module: creates a new user account. Login Module: authenticates the user. Complaint Module: accepts complaint details and stores them. Database Module:

manages CRUD operations. Deployment Module: hosts the application online for public access.

7. Project Workflow

The workflow begins when a new user registers. After successful authentication, the user logs in and accesses the complaint form. Complaint information is validated and sent to the backend. The backend stores the data in MongoDB and returns a success response. The user can then verify that the complaint has been submitted successfully. The workflow begins when a new user registers. After successful authentication, the user logs in and accesses the complaint form. Complaint information is validated and sent to the backend. The backend stores the data in MongoDB and returns a success response. The user can then verify that the complaint has been submitted successfully. The workflow begins when a new user registers. After successful authentication, the user logs in and accesses the complaint form. Complaint information is validated and sent to the backend. The backend stores the data in MongoDB and returns a success response. The user can then verify that the complaint has been submitted successfully. The workflow begins when a new user registers. After successful authentication, the user logs in and accesses the complaint form. Complaint information is validated and sent to the backend. The backend stores the data in MongoDB and returns a success response. The user can then verify that the complaint has been submitted successfully.

8. Testing

Functional testing verifies registration, login, and complaint submission. API testing is performed using Thunder Client or Postman. Integration testing confirms successful communication between frontend, backend, and database. Deployment testing ensures the application works after publishing. Functional testing verifies registration, login, and complaint submission. API testing is performed using Thunder Client or Postman. Integration testing confirms successful communication between frontend, backend, and database. Deployment testing ensures the application works after publishing. Functional testing verifies registration, login, and complaint submission. API testing is performed using Thunder Client or Postman. Integration testing confirms successful communication between frontend, backend, and database. Deployment testing ensures the application works after publishing. Functional testing verifies registration, login, and complaint submission. API testing is performed using Thunder Client or Postman. Integration testing confirms successful communication between frontend, backend, and database. Deployment testing ensures the application works after publishing.

9. Challenges and Solutions

During development, issues such as MongoDB connection errors, missing dependencies, API routing problems, and deployment configuration were encountered. These were resolved by checking environment variables, installing required packages, verifying API routes, and testing endpoints individually. During development, issues such as MongoDB connection errors, missing dependencies, API routing problems, and deployment configuration were encountered. These were resolved by checking environment variables, installing required packages, verifying API routes, and testing endpoints individually. During development, issues such as MongoDB connection errors, missing dependencies, API routing problems, and deployment configuration were encountered. These were resolved by checking environment variables, installing required packages, verifying API routes, and testing endpoints individually. During development, issues such as MongoDB connection errors, missing dependencies, API routing problems, and deployment configuration were encountered. These were resolved by checking environment variables, installing required packages, verifying API routes, and testing endpoints individually.

10. Future Enhancements

Future versions can include an administrator dashboard, complaint status updates, email notifications, SMS alerts, image attachments, advanced search filters, analytics dashboards, role-based authentication, and password recovery. Future versions can include an administrator dashboard, complaint status updates, email notifications, SMS alerts, image attachments, advanced search filters, analytics dashboards, role-based authentication, and password recovery.

Future versions can include an administrator dashboard, complaint status updates, email notifications, SMS alerts, image attachments, advanced search filters, analytics dashboards, role-based authentication, and password recovery. Future versions can include an administrator dashboard, complaint status updates, email notifications, SMS alerts, image attachments, advanced search filters, analytics dashboards, role-based authentication, and password recovery.

11. Documentary Script

Introduce yourself and the project. Explain the problem statement, objectives, technologies, folder structure, frontend, backend, MongoDB integration, API flow, GitHub repository, deployment, live demonstration, challenges, future scope, and conclude by summarizing how the project solves the problem. While speaking, demonstrate the corresponding screens in VS Code, terminal, browser, GitHub, and the deployed application. Introduce yourself and the project. Explain the problem statement, objectives, technologies, folder structure, frontend, backend, MongoDB integration, API flow, GitHub repository, deployment, live demonstration, challenges, future scope, and conclude by summarizing how the project solves the problem. While speaking, demonstrate the corresponding screens in VS Code, terminal, browser, GitHub, and the deployed application. Introduce yourself and the project. Explain the problem statement, objectives, technologies, folder structure, frontend, backend, MongoDB integration, API flow, GitHub repository, deployment, live demonstration, challenges, future scope, and conclude by summarizing how the project solves the problem. While speaking, demonstrate the corresponding screens in VS Code, terminal, browser, GitHub, and the deployed application. Introduce yourself and the project. Explain the problem statement, objectives, technologies, folder structure, frontend, backend, MongoDB integration, API flow, GitHub repository, deployment, live demonstration, challenges, future scope, and conclude by summarizing how the project solves the problem. While speaking, demonstrate the corresponding screens in VS Code, terminal, browser, GitHub, and the deployed application.

12. Conclusion

The Online Complaint Registration System demonstrates practical implementation of the MERN stack by integrating a responsive frontend, RESTful backend, and NoSQL database into one complete solution. The project provides valuable experience in full-stack development, debugging, deployment, and version control. The Online Complaint Registration System demonstrates practical implementation of the MERN stack by integrating a responsive frontend, RESTful backend, and NoSQL database into one complete solution. The project provides valuable experience in full-stack development, debugging, deployment, and version control. The Online Complaint Registration System demonstrates practical implementation of the MERN stack by integrating a responsive frontend, RESTful backend, and NoSQL database into one complete solution. The project provides valuable experience in full-stack development, debugging, deployment, and version control. The Online Complaint Registration System demonstrates practical implementation of the MERN stack by integrating a responsive frontend, RESTful backend, and NoSQL database into one complete solution. The project provides valuable experience in full-stack development, debugging, deployment, and version control.